# SIMATS ENGINEERING
## ASSESSMENT TOOL 3

**COURSE NAME: INTERNET PROGRAMMING**
**COURSE CODE: CSA43**

---

**COURSE OUTCOME COVERED**

**CO-1:** Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Code along exercise: Make your first HTML page together with CSS

**Weightage: 10%**

---

**QUESTIONS:** **Total Marks: 10**

## Scenario: Web Development Standards Implementation

A start up is developing a modern web application. During development, the team encounters multiple issues related to HTML elements, HTTP methods, XML syntax, and web communication protocols.
The following observations are made:

- The team needs to embed a promotional video on the homepage.
- A registration form must securely send user data to the server.
- The navigation menu must follow semantic HTML standards.
- XML configuration files are being used but contain syntax errors.
- The application must communicate over the correct web protocol.

## Data Snippets Provided

### Snippet 1: Video Embedding Options

```
A. <media src="video.mp4"></media>
B. <video src="video.mp4" controls></video>
```

```
C. <movie src="video.mp4"></movie>
D. <vid src="video.mp4"></vid>
```

**Snippet 2: Form Submission Methods**

```
A. GET
B. POST
C. PUT
D. DELETE
```

**Snippet 3: Navigation Elements**

```
A. <div>
B. <nav>
C. <section>
D. <header>
```

**Snippet 4: XML Syntax**

```
A. <note><to>User</to><from>Admin</from>
B. <note><to>User</to><from>Admin</from></note>
C. <note><to>User</from></note>
D. <note to="User"></note
```

**Snippet 5: Web Communication Protocols**

```
A. FTP
B. HTTP
C. SMTP
D. SNMP
```

## Questions:

1. Select the **correct HTML tag for video embedding** and justify your answer.
2. Identify the **appropriate HTTP method for form submission** and explain why.
3. Choose the **proper semantic element for navigation** and justify its usage.
4. Identify the **correct XML syntax** and explain the error in other options.
5. Choose the **correct protocol for web communication** and justify your choice.

---

***Code of Conduct:***

*I YARRAPUREDDY LAKSHMI REDDY (192324077) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: Yarrapureddy Lakshmi Reddy***

## RUBRICS FOR CALCULATION:

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Case | 25 | Thoroughly understands the scenario; correctly identifies the requirement behind all five observations | Understands most requirements of the scenario | Basic understanding; some requirements missed | Poor understanding; requirements not identified |
| Application of Concepts | 25 | Correctly applies HTML5, HTTP, semantic-HTML and XML concepts to choose the right option in every snippet | Applies concepts correctly with minor gaps | Limited application; some wrong options chosen | Fails to apply the relevant concepts |
| Analysis | 20 | Justifies the correct option and explains why every other option is wrong, with strong reasoning | Good justification with logical reasoning | Basic justification with limited depth | Weak or no justification |
| Solution Design | 20 | Working HTML + CSS code and output screenshots that clearly demonstrate every answer | Correct code and outputs with minor issues | Basic code or few outputs, weak support | Incorrect or no code or output provided |
| Clarity | 10 | Clear, well-structured answers with labelled code and figures | Organized and understandable presentation | Limited clarity and structure | Poor clarity; difficult to understand |

# ANSWERS

**Name: Yarrapureddy Lakshmi Reddy  Reg No: 192324077**

**Summary of answers:** Q1 – B (<video>) | Q2 – B (POST) | Q3 – B (<nav>) | Q4 – B (well-formed XML) | Q5 – B (HTTP, secured with HTTPS).

Note on outputs: the screenshots are real outputs produced by running the snippets on a local test setup (a browser engine for rendering, xmllint for XML, curl against a local test server).

## Question 1: Correct HTML tag for video embedding

**Answer: Option B – <video src="video.mp4" controls></video>**

### Justification

- **<video>** is the standard HTML5 element for embedding video. It plays the file natively in the browser without any plug-in such as Flash.
- The **src** attribute gives the file (mp4/H.264, WebM or Ogg) and the Boolean attribute **controls** shows the play, pause, seek and volume buttons.
- Text placed between the opening and closing tag is shown as fallback content in browsers that do not support video.

### Why the other options are wrong

**<media>**, **<movie>** and **<vid>** are not defined in the HTML specification. A browser treats them as unknown elements, ignores them and shows nothing, so the promotional video never appears (Figure 1).

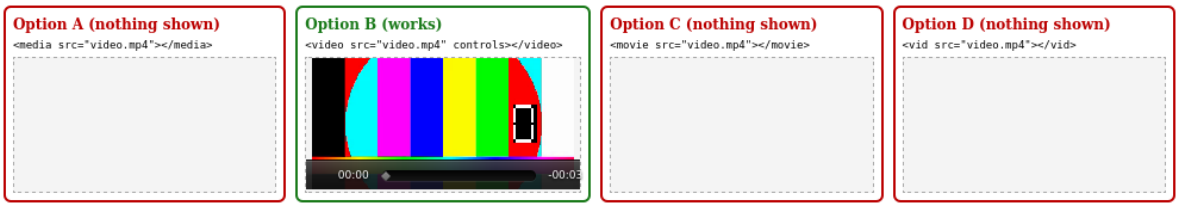



*Figure 1: only Option B produces a working video player; A, C and D render nothing*

```
<video src="video.mp4" controls width="480" poster="poster.jpg">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Your browser does not support the video tag.
</video>
```

Improved version: several <source> formats, a poster image and fallback text.

# Question 2: Appropriate HTTP method for form submission

**Answer: Option B – POST**

## Justification

- POST places the form data in the **request body**, so the username and password are not shown in the URL, browser history, bookmarks or server access logs.
- There is no practical size limit, and POST is meant for requests that create or change data on the server, such as registering a new user.
- POST is not cached and is not re-sent silently, so a registration is not repeated by accident.

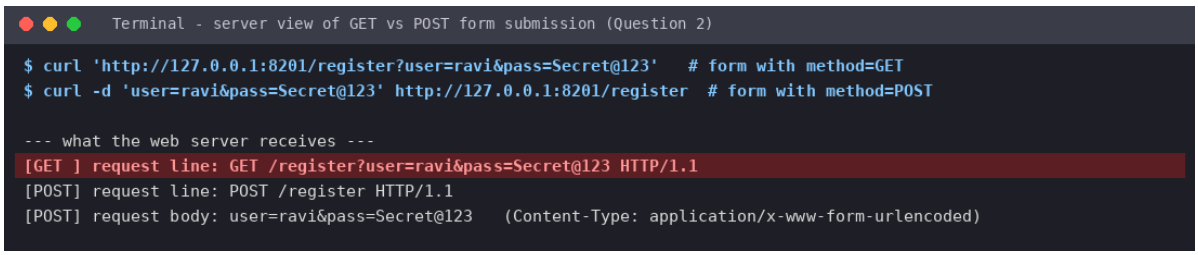



*Figure 2: with GET the password appears in the request line; with POST it travels in the body*

## Why the other options are wrong

- **GET** appends the data to the URL (?user=..&pass=..), which exposes the credentials, is limited in length and can be cached. It is meant for retrieving data.
- **PUT** replaces or updates a resource at a known URL and **DELETE** removes a resource. Both are used by REST APIs, and HTML forms cannot send them (a form supports only GET and POST).

POST alone does not encrypt the data. To send it **securely**, the form must be served over **HTTPS** and the server must hash passwords, validate input and use CSRF protection.

```
<form action="/register" method="POST">
  <label for="user">Username</label>
  <input type="text" id="user" name="user" required>
  <label for="pass">Password</label>
  <input type="password" id="pass" name="pass" required>
  <button type="submit">Register</button>
</form>
```

# Question 3: Proper semantic element for navigation

**Answer: Option B – <nav>**

## Justification

- **<nav>** is the HTML5 semantic element for a block of major navigation links, such as the main menu.
- Browsers and screen readers expose it as a **navigation landmark**, so users with assistive technology can jump straight to the menu or skip it (Figure 3).
- Search engines understand the page structure better, and the code is easier to read and maintain than a pile of anonymous  elements.

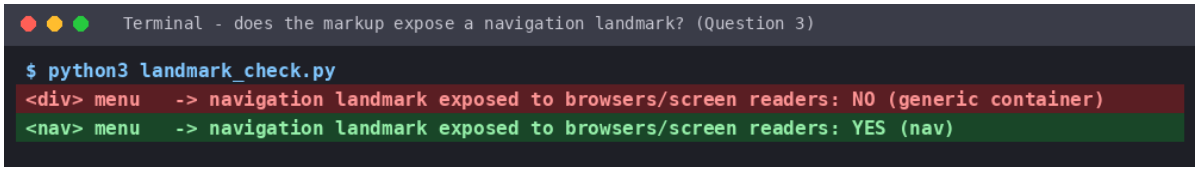



*Figure 3: a <nav> menu is recognised as a navigation landmark, a  menu is not*

## Why the other options are wrong

- ** ** has no meaning; it is only a generic container used for styling.
- **<section>** groups thematic content, normally with its own heading; it does not mean navigation.
- **<header>** holds introductory content such as the logo and title. It may contain a <nav>, but it is not itself the navigation.

```
<nav>
  <ul>
    <li><a href="#home">Home</a></li>
    <li><a href="#video">Promo Video</a></li>
    <li><a href="#register">Register</a></li>
  </ul>
</nav>
```

# Question 4: Correct XML syntax

**Answer: Option B – <note><to>User</to><from>Admin</from></note>**

## Justification

An XML document must be well-formed: it has one root element, every start tag has a matching end tag, elements are properly nested, names are case-sensitive and attribute values are quoted. Option B follows all these rules, and xmllint accepts it without errors (Figure 4).

## Error in the other options

| Option | Error |
|---|---|
| A | The root element <note> is never closed – the end tag </note> is missing (“Premature end of data”). |
| C | <to> is closed with </from>, so the start and end tags do not match and the nesting is broken. |
| D | The closing tag is incomplete: </note has no “>” (“expected '>'”). The attribute to="User" itself is valid XML. |

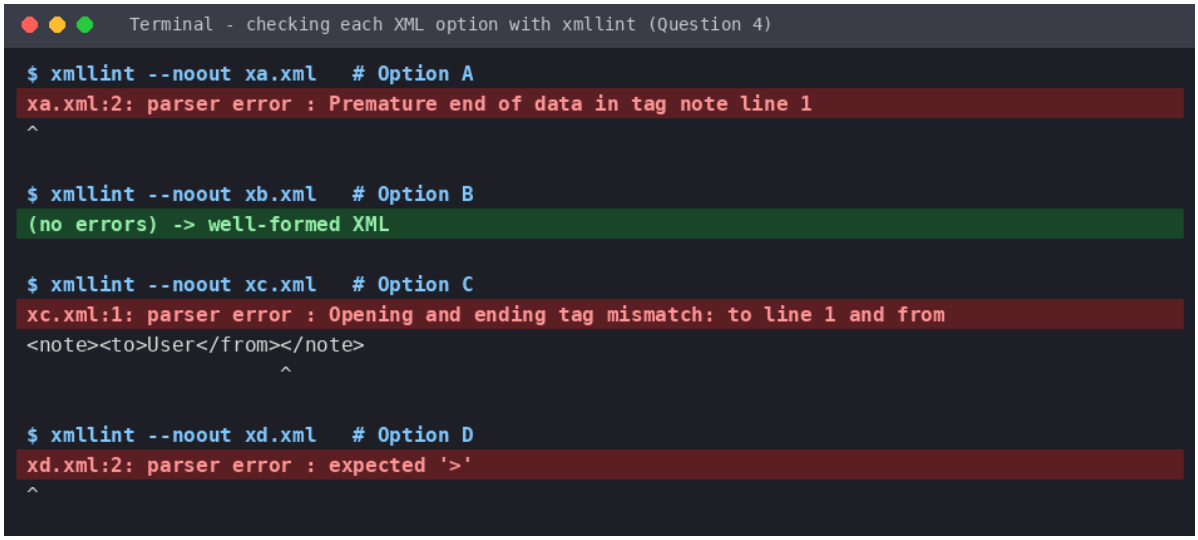



*Figure 4: xmllint accepts Option B and reports the exact error in A, C and D*

# Question 5: Correct protocol for web communication

**Answer: Option B – HTTP** (secured as HTTPS)

## Justification

- **HTTP** (HyperText Transfer Protocol) is the application-layer protocol used between a web browser and a web server. The browser sends a request such as GET /index.html and the server returns a response with a status code, headers and the page (Figure 5).
- It runs over TCP on port **80**. **HTTPS** is HTTP inside TLS on port **443**; it should be used for the registration form so that the data is encrypted.

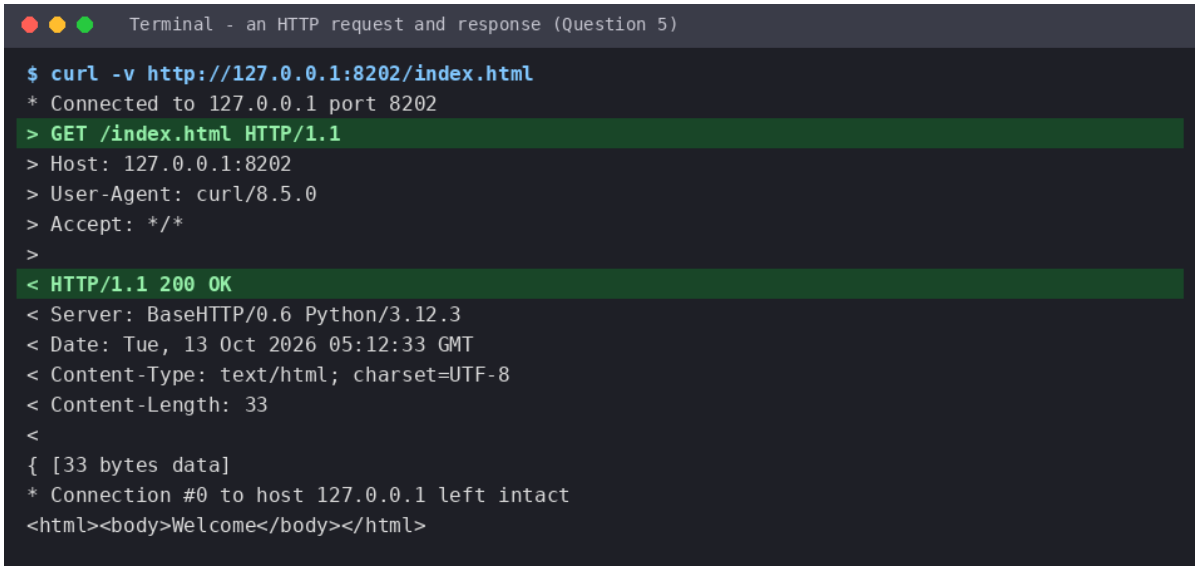



*Figure 5: an HTTP request (GET) and response (200 OK) captured with curl*

## Why the other options are wrong

| Protocol | Purpose | Default port |
|---|---|---|
| FTP | Transferring files between computers | 20 / 21 |
| SMTP | Sending e-mail between mail servers | 25 / 587 |
| SNMP | Monitoring and managing network devices | 161 / 162 (UDP) |
| HTTP | Requesting and delivering web pages (web communication) | 80 (443 for HTTPS) |

## Code-Along Exercise: First HTML Page with CSS

The complete page below puts all five answers together: a semantic <nav>, an embedded <video>, and a registration form that uses POST, styled with CSS.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>StartUp Web App</title>
<style>
 body { margin:0; font-family: Arial, sans-serif; background:#f4f6fa; color:#222; }
 header { background:#1f3864; color:#fff; padding:14px 24px; }
 header h1 { margin:0 0 8px; font-size:22px; }
 nav a { color:#fff; text-decoration:none; margin-right:18px; font-size:14px; padding:4px 8px; background:#2e4f8f; border-radius:4px; }
 main { padding:18px 24px; }
 section { background:#fff; border:1px solid #ccd; border-radius:6px; padding:12px 16px; margin-bottom:14px; }
 h2 { margin:0 0 8px; font-size:17px; color:#1f3864; }
 video { width:360px; height:180px; background:#000; display:block; }
 label { display:block; margin:6px 0 2px; font-size:13px; }
 input { padding:5px; width:240px; }
 button { margin-top:10px; padding:6px 16px; background:#1f3864; color:#fff; border:0; border-radius:4px; }
</style>
</head>
<body>
<header>
 <h1>StartUp Web App</h1>
 <nav>
  <a href="#home">Home</a>
  <a href="#video">Promo Video</a>
  <a href="#register">Register</a>
  <a href="#contact">Contact</a>
 </nav>
</header>
<main>
 <section id="video">
  <h2>Promotional Video</h2>
  <video src="video.mp4" controls></video>
 </section>
 <section id="register">
  <h2>Register</h2>
  <form action="/register" method="POST">
   <label for="user">Username</label>
   <input type="text" id="user" name="user" required>
   <label for="pass">Password</label>
   <input type="password" id="pass" name="pass" required>
   <button type="submit">Register</button>
```

```
  </form>
 </section>
</main>
</body>
</html>
```

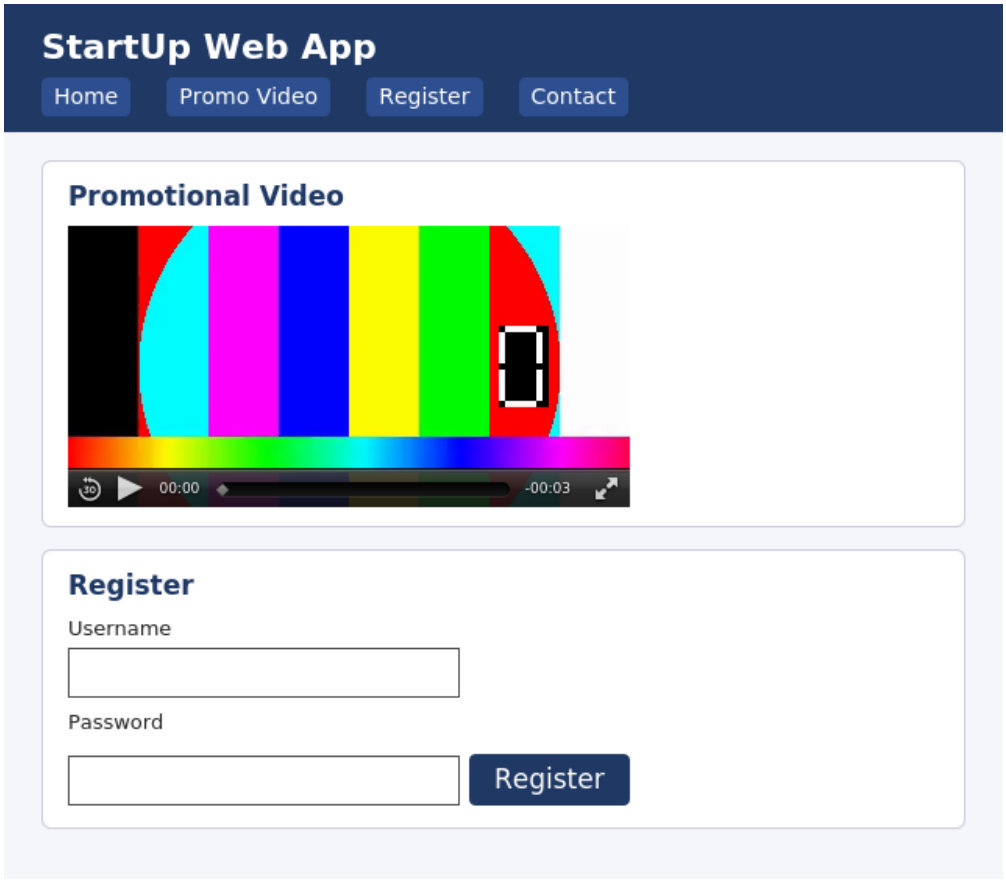



*Figure 6: browser output of the complete page (nav bar, video player and POST registration form)*

**Overall conclusion:** the correct choices are the <video> element for video, POST for form data, <nav> for navigation, well-formed XML with matching tags, and HTTP (preferably HTTPS) for web communication. Using these standards makes the application correct, accessible and secure.